

AGENTIC RAG | PRODUCTION CLOUD GUIDE

Agentic RAG — AWS Deployment Guide

Service catalog and step-by-step production deployment

Application: Canonical Agentic RAG (FastAPI + React + Redis + ChromaDB)

Pipeline: canonical_pipeline_version = v1

Document date: 8 September 2026

Cloud: AWS

What this document covers

This guide maps the Agentic RAG repository onto a production-shaped AWS topology. It lists every cloud service you need, why it exists, and a concrete ordered procedure to build, configure, ingest, and verify the stack. It is written against the current code: FastAPI in `src/api/server.py`, the React/nginx frontend, Redis cache and rate limits, Chroma HTTP, and the production boot checks in `ENVIRONMENT=production`.

Recommended path on AWS is Amazon ECS on Fargate plus ElastiCache, EFS, and an ALB. A single EC2 + Docker Compose box is documented as staging only.

Contents

1. What you are deploying
2. Complete service catalog (required, recommended, optional)
3. Recommended topology
4. Prerequisites
5. Step-by-step deployment
6. Verification
7. Scaling, cost, and limits
8. Simpler staging path
9. Troubleshooting
10. Production checklist

1. What you are deploying

Agentic RAG is not a notebook. Production is a multi-container service: a FastAPI agent runtime, a React UI that reverse-proxies `/api` and injects X-API-Key server-side, Redis for answer cache / rate limits / idempotency, and a Chroma HTTP server for embeddings. Optional pieces are Prometheus/Grafana, LangSmith tracing, NVIDIA rerank, Groq LLM failover, and Supabase (or a managed Postgres) for conversation memory and thumbs-up/down feedback.

Workload	Image / runtime	Port	Notes
API	Repo Dockerfile (Python 3.10, uvicorn)	8000	Liveness GET <code>/health</code> . Readiness <code>/health/ready</code> is auth-gated. SSE on POST <code>/query/stream</code> (idle up to 300s).
Frontend	frontend/Dockerfile (nginx 1.27)	80	Proxies <code>/api/</code> to the API. Injects API_KEY. SSE buffering must stay off. Upload body 25 MB.
Redis	redis:7 (managed service in cloud)	6379	Required in production for shared rate limits and cache. Password/auth token mandatory.
Chroma	chromadb/chroma:0.6.3	8000	Set CHROMA_MODE=http. Persist the volume. Do not publish this port to the internet.
Docs / SQLite	Shared file volume	—	PDFs, parent store, extra-source SQLite, eval DBs. In-process BM25 lives in the API.

Production boot will refuse to start unless all of the following are true:

- OPENAI_API_KEY is set.
- API_KEY is set and at least 32 characters (`openssl rand -hex 32`).
- CORS_ORIGINS lists real frontend origins — `*` is rejected.
- API_WORKERS > 1 only if RATE_LIMIT_BACKEND=redis.
- TRUST_CLIENT_RBAC stays false.

Scale-out constraint. BM25, the semantic cache, and in-process ingest jobs are not shared across replicas. Keep API_WORKERS=1 and a single API task until you move hybrid search to a managed vector store. Size MAX_CONCURRENT_QUERIES against your OpenAI concurrency, not against CPU.

External systems the app already calls (not provisioned in the VPC):

System	Used for	Required?
OpenAI	Chat completions + text-embedding-3-small	Yes
NVIDIA NIM (build.nvidia.com)	Cross-encoder rerank if RERANK_PROVIDER=nvidia	Recommended
Groq	LLM failover when OpenAI fails	Optional
LangSmith	Parent-span traces <code>agent_request: <mode></code>	Optional
DuckDuckGo	Web fallback in the graph (circuit-broken)	Optional
Supabase	Persistent chat memory + answer_feedback	Optional; replace with managed Postgres

Environment variables to set on the API

Variable	Production value
ENVIRONMENT	production
OPENAI_API_KEY	from secrets manager / Key Vault
API_KEY	same 32+ char secret injected into frontend nginx
REQUIRE_API_KEY	true
CORS_ORIGINS	https://rag.example.com
TRUSTED_HOSTS	rag.example.com
TRUST_PROXY_HEADERS	true (TLS terminates at the load balancer)
CHROMA_MODE	http
CHROMA_HOST / CHROMA_PORT	chroma.rag.local / 8000
REDIS_URL	rediss://:\${REDIS_AUTH}@rag-redis.cache.amazonaws.com:6379/0
CACHE_ENABLED	true
RATE_LIMIT_BACKEND	redis
API_WORKERS	1
PROTECT_METRICS_ENDPOINT	true unless scraper is on a private network
PROTECT_READINESS_ENDPOINT	true — load balancer probes /health only
NVIDIA_API_KEY	if rerank stays on NVIDIA
GROQ_API_KEY	optional failover
INGEST_ALLOWED_ROOTS	/app/data
REQUEST_TIMEOUT_SECONDS	120
STREAM_TIMEOUT_SECONDS	300
MAX_CONCURRENT_QUERIES	8 (tune after a load test)

Frontend container needs `API_UPSTREAM` (private API URL) and the **same** `API_KEY`. The browser never holds the key; nginx injects `X-API-Key` and strips any client-supplied one.

2. Complete AWS service catalog

2.1 Required services

AWS service	Role in this app	Maps to compose
Amazon VPC + 2 AZs	Private API/Redis/Chroma; public ALB only	compose network
Internet Gateway + NAT Gateway	Inbound HTTPS; outbound OpenAI/NVIDIA/Groq	host egress
Security groups	ALB 443; frontend 80 from ALB; API 8000 from frontend SG; Redis/Chroma internal	expose vs ports
Amazon ECR	Store API and frontend images built from this repo	docker build
Amazon ECS (Fargate)	Run API, frontend, and Chroma tasks	compose services
AWS Cloud Map	Private DNS: api.rag.local, chroma.rag.local	service names
Elastic Load Balancing (ALB)	HTTPS, host routing, SSE idle timeout 360s	published 8080/8000
AWS Certificate Manager	Public TLS cert for the app hostname	uncomment HSTS later
Amazon Route 53	A/AAAA alias to the ALB	local hosts file
Amazon ElastiCache (Redis 7)	Cache, rate limits, idempotency keys	redis service
Amazon EFS	Chroma persistence + /app/data (PDFs, SQLite, parent store)	named volumes
Amazon S3	Source corpus, backups of chroma/parent_store	data/sample_docs
AWS Secrets Manager	OPENAI_API_KEY, API_KEY, REDIS auth, NVIDIA, Groq, LangSmith	.env.production
Amazon CloudWatch Logs	JSON stdout from API (production logging)	compose logs
Amazon CloudWatch Alarms	5xx, unhealthy hosts, Redis CPU, OpenAI error spikes	Grafana optional
IAM roles	Task execution (pull image, read secrets) + task role (S3/EFS)	container user

2.2 Strongly recommended

AWS service	Why
AWS WAF on the ALB	Rate, bot, and SQLi/XSS managed rules in front of nginx
AWS CloudTrail	Audit who changed the cluster, secrets, and security groups
Amazon GuardDuty	Threat detection on the account
AWS Backup	EFS + S3 backup vault with retention
Amazon RDS PostgreSQL	Replace Supabase for chat_messages and answer_feedback (RLS still your job)
Amazon CloudFront	Optional CDN for hashed /assets/; keep /api on ALB with streaming
AWS Managed Prometheus + Amazon Managed Grafana	Scrape /metrics privately; import the repo dashboard
AWS Budgets / Cost Anomaly Detection	LLM spend is unbounded without MAX_TOKENS_* plus an account budget

2.3 Optional / later

AWS service	When to add it
Amazon EKS	Only if you already run Kubernetes; Fargate ECS matches compose with less ops
Amazon OpenSearch / Bedrock Knowledge Bases	When BM25-in-process hits BM25_MAX_DOCS (20k)
Amazon SQS + extra ingest worker	When PDF batches exceed the in-process ingest queue
AWS App Runner	Not recommended: no private Redis/Chroma sidecar story
Amazon EC2 (single host)	Cheap staging: run the existing docker-compose.yml unchanged

3. Recommended topology

Use **ECS on Fargate** behind an internet-facing ALB. Put Redis and Chroma in private subnets. The React container is the only public target. Nginx talks to the API over Cloud Map. The API talks to ElastiCache and Chroma on the same VPC. NAT Gateway is required so Fargate tasks can reach OpenAI, NVIDIA, and Groq without public IPs.

```

Internet
  ■ HTTPS :443 (ACM cert, WAF optional)
  ▼
Application Load Balancer  idle timeout = 360s
  ■ target group: frontend :80
  ▼
ECS Fargate [frontend/nginx]
  ■ API_UPSTREAM=http://api.rag.local:8000
  ■ API_KEY from Secrets Manager
  ▼
ECS Fargate [agentic-rag API :8000]  ■■ outbound via NAT ■■■ OpenAI / NVIDIA / Groq / LangSmith
  ■
  ■■■ ElastiCache Redis (auth token, encryption in transit)
  ■■■ ECS Fargate [chroma :8000] + EFS /chroma/chroma
  ■■■ EFS /app/data (PDFs, parent_store.json, knowledge.db, feedback.db)

S3 bucket rag-corpus (source PDFs; copy into EFS or ingest from a one-off task)
Secrets Manager /agentic-rag/prod/*
CloudWatch Logs /ecs/agentic-rag-api
  
```

Do **not** register the API, Redis, or Chroma as public ALB targets. The compose stack already follows this rule: only frontend 8080 and API 8000 are published locally, and even the API should stay private in AWS because nginx is the public edge.

4. Prerequisites

- AWS account with billing alerts. Region example below: us-east-1.
- IAM principal that can create VPC, ECS, ECR, ElastiCache, EFS, ALB, ACM, Route 53, Secrets Manager.
- AWS CLI v2, Docker Desktop, and this git repo cloned.
- A DNS name you control (e.g. rag.example.com).
- OpenAI key; NVIDIA key if you keep cloud rerank; 32+ character app API key.
- Local sanity check already green: `pytest -q` and a Docker build of both images.

```

export AWS_REGION=us-east-1
export AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export APP_HOST=rag.example.com
export API_KEY=$(openssl rand -hex 32)
aws configure list
  
```

5. Step-by-step deployment

Step 1 — Create the network

Create a VPC with two public and two private subnets across two AZs, an internet gateway, and a NAT Gateway in each AZ (or one NAT for cost, accepting the AZ risk). Tag it `agentic-rag-prod`. Enable DNS hostnames.

Console path: VPC → Create VPC → VPC and more → 2 AZs, 2 public, 2 private, NAT gateways. CLI users can use a CloudFormation/Terraform module; the exact IDs are referenced as `$VPC_ID` `$PUBLIC_SUBNETS` `$PRIVATE_SUBNETS` below.

```
# Security groups (create after VPC exists)
aws ec2 create-security-group --group-name rag-alb-sg --description "ALB HTTPS" --vpc-id $VPC_ID
aws ec2 authorize-security-group-ingress --group-id $ALB_SG --protocol tcp --port 443 --cidr 0.0.0.0/0

aws ec2 create-security-group --group-name rag-frontend-sg --description "nginx from ALB" --vpc-id $VPC_ID
aws ec2 authorize-security-group-ingress --group-id $FE_SG --protocol tcp --port 80 --source-group $ALB_SG

aws ec2 create-security-group --group-name rag-api-sg --description "API from frontend" --vpc-id $VPC_ID
aws ec2 authorize-security-group-ingress --group-id $API_SG --protocol tcp --port 8000 --source-group $FE_SG

aws ec2 create-security-group --group-name rag-data-sg --description "Redis+Chroma from API" --vpc-id $VPC_ID
aws ec2 authorize-security-group-ingress --group-id $DATA_SG --protocol tcp --port 6379 --source-group $API_SG
aws ec2 authorize-security-group-ingress --group-id $DATA_SG --protocol tcp --port 8000 --source-group $API_SG
```

Step 2 — Container registry and images

Create two ECR repositories. Chroma can keep using the public `chromadb/chroma:0.6.3` image (pin the digest in the task definition) or you can mirror it into ECR.

```
aws ecr create-repository --repository-name agentic-rag-api --image-scanning-configuration scanOnPush=true --region $AWS_REGION
aws ecr create-repository --repository-name agentic-rag-frontend --image-scanning-configuration scanOnPush=true --region $AWS_REGION

aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION

docker build -t $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/agentic-rag-api:v1 .
docker build -t $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/agentic-rag-frontend:v1 ./frontend

docker push $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/agentic-rag-api:v1
docker push $AWS_ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com/agentic-rag-frontend:v1
```

The API Dockerfile is multi-stage, non-root (appuser), and CI already fails if a `.env` is baked into the image. Never pass secrets as Docker ARG/ENV at build time.

Step 3 — Secrets

```
aws secretsmanager create-secret --name agentic-rag/prod --secret-string "$(cat <<EOF
{
  "OPENAI_API_KEY": "sk-...",
  "API_KEY": "${API_KEY}",
  "NVIDIA_API_KEY": "nvapi-...",
  "GROQ_API_KEY": "",
  "LANGSMITH_API_KEY": "",
  "REDIS_AUTH_TOKEN": "$(openssl rand -hex 24)"
}
EOF
)"
```

ECS task definitions should reference this secret with `valueFrom` per key. The frontend task only needs `API_KEY`.

Step 4 — ElastiCache Redis

Create a Redis 7 replication group in the private subnets, subnet group on `$DATA_SG`, transit encryption on, at-rest encryption on, AUTH token from the secret. Disable public access. Start with `cache.t4g.small`; this app uses Redis as a cache and token bucket, not as a primary database.

```
# REDIS_URL the API expects (TLS example):
# rediss://:${REDIS_AUTH_TOKEN}@rag-redis.xxxxxx.cache.amazonaws.com:6379/0
#
# CACHE_ENABLED=true
# RATE_LIMIT_BACKEND=redis
```

Step 5 — EFS for Chroma and application data

Create one EFS filesystem with mount targets in both private subnets, using \$DATA_SG (add NFS 2049 from the API and Chroma security groups). Create two access points:

- /chroma — uid 1000, mounted at /chroma/chroma on the Chroma task.
- /appdata — uid 1000, mounted at /app/data on the API task (PDFs, parent_store, SQLite).

Upload the sample corpus (or your PDFs) to S3, then copy onto EFS with a one-off Fargate task or AWS DataSync. The ingest CLI reads from INGEST_ALLOWED_ROOTS=/app/data.

Step 6 — ECS cluster, namespace, and log groups

```
aws ecs create-cluster --cluster-name agentic-rag-prod --capacity-providers FARGATE FARGATE_SPOT \
  --default-capacity-provider-strategy capacityProvider=FARGATE,weight=1

aws servicediscovery create-private-dns-namespace --name rag.local --vpc $VPC_ID --region $AWS_REGION

aws logs create-log-group --log-group-name /ecs/agentic-rag-api
aws logs create-log-group --log-group-name /ecs/agentic-rag-frontend
aws logs create-log-group --log-group-name /ecs/agentic-rag-chroma
```

Create an IAM task execution role with AmazonECSTaskExecutionRolePolicy plus secretsmanager:GetSecretValue on agentic-rag/prod. Create a task role with EFS client and S3 read on the corpus bucket.

Step 7 — Task definitions (order: Chroma → API → frontend)

Chroma task — 1 vCPU / 2 GB, image chromadb/chroma:0.6.3, port 8000, EFS mount, env:

```
IS_PERSISTENT=TRUE
ANONYMIZED_TELEMETRY=FALSE
# Service discovery name: chroma.rag.local
```

API task — 1 vCPU / 4 GB (embeddings + BM25 are memory-heavy), port 8000, EFS /app/data. Health check:

```
CMD-SHELL, python -c "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://localhost:8000/health', ti
```

Command stays the image default: uvicorn on 8000, API_WORKERS=1. CPU/memory: start 1024 CPU / 4096 MB. Hybrid retrieval loads chunks into RAM.

Frontend task — 0.25 vCPU / 0.5 GB, port 80, env:

```
API_UPSTREAM=http://api.rag.local:8000
API_KEY=<from Secrets Manager>
```

Step 8 — Application Load Balancer

- 1 Create an internet-facing ALB in the public subnets, security group \$ALB_SG.
- 2 Create a target group rag-frontend, type IP, port 80, protocol HTTP, VPC private targets.
- 3 Health check path / or /index.html, matcher 200.
- 4 Request an ACM certificate for \$APP_HOST (DNS validation in Route 53).
- 5 HTTPS listener 443 → forward to rag-frontend. HTTP 80 → redirect to 443.
- 6 **Idle timeout = 360 seconds** (Attributes). Default 60s will kill SSE streams (STREAM_TIMEOUT_SECONDS=300 plus nginx 320s).

7 Stickiness off. HTTP/2 on. Drop invalid headers on.

If you later expose the API on a second host (api.example.com), create a second target group on port 8000 and probe /health only — never /health/ready, which is auth-gated and returns 401 to the load balancer.

Step 9 — ECS services

Create three Fargate services on private subnets, assign public IP **disabled**, use the matching security groups. Desired count: Chroma 1, API 1, frontend 2. Frontend can scale; API should stay at 1 replica until BM25 is externalized. Attach the frontend service to the ALB target group.

```
# After services are RUNNING:
aws ecs update-service --cluster agentic-rag-prod --service frontend --desired-count 2
aws ecs update-service --cluster agentic-rag-prod --service api --desired-count 1
aws ecs update-service --cluster agentic-rag-prod --service chroma --desired-count 1
```

Step 10 — DNS

In Route 53, alias A record \$APP_HOST → ALB. Wait for ACM + DNS. Then set:

```
CORS_ORIGINS=https://rag.example.com
TRUSTED_HOSTS=rag.example.com
TRUST_PROXY_HEADERS=true
```

Redeploy the API after those three values change. Uncomment the HSTS header in frontend/nginx.conf once you confirm TLS is terminating in front of nginx, rebuild, and push a new frontend image.

Step 11 — Ingest the knowledge base

Chroma starts empty. Run a one-off ECS task (same API image, same EFS, same secrets) that overrides the command:

```
python -m src.ingestion.ingest --source /app/data/sample_docs
```

Confirm readiness from a jump host or ECS exec (send the API key):

```
curl -sS https://rag.example.com/api/health
curl -sS -H "X-API-Key: $API_KEY" https://rag.example.com/api/health/ready
# Expect chroma check "ok" with a non-zero document count.
```

Ongoing uploads go through the UI (POST /ingest/upload) or POST /ingest/jobs. Re-ingest flushes Redis answer keys (rag:v1:*) so stale answers are not served.

Step 12 — Observability and alarms

- CloudWatch Logs metric filters on error_code and Token usage.
- ALB alarms: HTTPCode_Target_5XX, UnHealthyHostCount, TargetResponseTime p95.
- ElastiCache: EngineCPUUtilization, Evictions, CurrConnections.
- Optional: Amazon Managed Prometheus scrape of http://api.rag.local:8000/metrics with PROTECT_METRICS_ENDPOINT=false only on the private network (compose does this today).
- LangSmith: set LANGSMITH_TRACING=true and the project name; traces wrap classify → retrieve → generate.

Step 13 — CI/CD

Keep GitHub Actions CI as the quality gate (.github/workflows/ci.yml: ruff, pytest, pip-audit, gitleaks, frontend build, Docker image assertions). Add a **deploy** workflow on the prod branch:

- 1 OIDC to AWS (no long-lived access keys).
- 2 Build and push API + frontend images tagged with the git SHA.

- 3 Register new ECS task definitions.
- 4 Update frontend and API services (rolling). Chroma usually stays put.
- 5 Run the ingest one-off task only when the corpus or chunking config changed.
- 6 Smoke: GET /api/health and a canonical POST /api/query.

6. Verification

```
# Liveness (no auth)
curl -i https://rag.example.com/api/health

# Chat via the public UI origin (nginx injects the key)
curl -N https://rag.example.com/api/query/stream \
  -H "Content-Type: application/json" \
  -d '{"question":"What is Self-RAG?","mode":"canonical"}'

# Expect SSE frames: step, token, answer, follow_ups, sources, done
# Browser: open https://rag.example.com, ask a question, confirm citations render.
```

Also hit a cacheable question twice and confirm a `cache_hit` step on the second call. Then run a short Locust test from a jump host against staging, not production (`tests/load/locustfile.py`).

7. Scaling, cost, and limits

Knob	Guidance
Frontend tasks	Safe to autoscale 2–8 on ALB request count
API tasks	Keep 1 until hybrid search is a shared store; then Redis-backed rate limits allow N
MAX_CONCURRENT_QUERIES	Per replica. Total in-flight LLM calls = replicas × this value
Chroma	Single writer. Snapshot EFS. Vertical scale memory before adding replicas
Redis	TTL 3600s for answers; watch evictions after ingest flushes
ALB idle timeout	Must stay ≥ stream timeout or users see truncated SSE

Order-of-magnitude monthly cost for a small production (us-east-1, light traffic): NAT Gateway dominates networking; Fargate 1×1vCPU/4GB API + 2×0.25 vCPU frontend + Chroma; ElastiCache t4g.small; ALB; EFS; Secrets Manager. LLM tokens on OpenAI will usually exceed the AWS bill. Set OpenAI project limits and the app token budgets together.

8. Simpler staging path (EC2 + Compose)

If you only need a single-AZ staging box: launch Amazon Linux 2023 in a public subnet, install Docker, copy the repo, fill `.env.production` and `compose .env` (`REDIS_PASSWORD`, `API_KEY`, `GRAFANA_ADMIN_PASSWORD`), put an ALB or nginx with ACM in front of host port 8080, and run:

```
cp .env.production.example .env.production # chmod 600
# shell .env next to docker-compose.yml with REDIS_PASSWORD, API_KEY, GRAFANA_ADMIN_PASSWORD
docker compose up -d --build
python -m src.ingestion.ingest --source data/sample_docs # or exec in the api container
```

This is the same topology as `docs/PRODUCTION.md`. Promote to ECS when you need multi-AZ, managed Redis, and a deploy pipeline that does not SSH.

9. Troubleshooting

Symptom	Likely cause	Fix
API task never healthy	Missing API_KEY / CORS=* / no OPENAI key	Read CloudWatch: Unsafe production configuration
ALB 504 on long chats	Idle timeout 60s	Set ALB idle timeout 360s; nginx already 320s
401 on every UI call	Frontend API_KEY $\neq$ API API_KEY	One secret, two task definitions
Empty answers / no sources	Chroma empty	Run ingest one-off; check /health/ready
429 immediately	Redis unreachable so budgets mis-share, or limits too low	Check REDIS_URL; RATE_LIMIT_BACKEND=redis
Slow first query	Cold embeddings + NVIDIA rerank RTT	Warm with a canary question after deploy
OOM on API	BM25 loaded full corpus	Raise task memory or lower BM25_MAX_DOCS / move to OpenSearch

10. Production checklist

- Secrets only in Secrets Manager; images contain no .env.
- Redis AUTH + encryption in transit; security groups least-privilege.
- Chroma and Redis not on the ALB.
- CORS and TRUSTED_HOSTS equal the real https origin.
- ALB idle timeout 360s; frontend HSTS enabled after TLS verified.
- CloudWatch alarms on 5xx, unhealthy hosts, Redis CPU.
- Ingest completed; /health/ready chroma ok.
- OpenAI billing cap + app MAX_TOKENS_PER_HOUR.
- Backup: EFS and S3. Restore drill once.
- Load test on staging. Locust file in tests/load.
- WAF associated. CloudTrail on. Budget alert on.

Related repo docs: docs/PRODUCTION.md, docs/ARCHITECTURE.md, docs/GUARDRAILS.md, docs/API.md.